

👁 Watch

0

🔗 Fork

0

☆

Final Project for CS4092

☆ 0 stars 🔗 0 forks 👁 0 watching 🔗 1 branch 🏷 0 tags ↻ Activity

🌐 Public repository

🔗

1 Branch

🏷 0 Tags

🔗

🏷

Q Go to file

T

Go to file

Add file ➕

🔄

<> Code

⋮

 KrishKalathiya	Updated Project layout	04c6ad3 · 9 minutes ago	🕒
📁 db	Query 7	2 days ago	
📁 screens	fixed textual ui error	2 days ago	
📄 .gitignore	add TUI screens	2 days ago	
📄 DB_Proj_ER_Diags.excalidraw	Added Excalidraw file for ER Diagram	5 days ago	
📄 README.md	Updated Project layout	9 minutes ago	
📄 app.py	fixed textual ui error	2 days ago	
📄 queries.sql	Query 7	2 days ago	
📄 requirements.txt	add deps and gitignore	2 days ago	

README

E-Commerce Database System Requirements Document

Project layout

Path	Contents
db/schema.sql	Database Implementation - Table definitions and sample data
db/connection.py	Business Logic - SQLite connection handling
db/queries.py	Database Interaction - Every read query
db/writes.py	Database Interaction - Inserts, updates, deletes, and the checkout transaction
screens/	Business Logic - One module per tab
app.py	Business Logic - Application entry point

Path	Contents
queries.sql	Database Implementation - The queries as standalone SQL
README.md	Requirements Gathering - Project Management and Setup
DB_Proj_ER_Diagrams.pdf	ER Diagram - Exported PDF
DB_Schemas.pdf	Schema Design - Exported PDF

1. System Overview

The purpose of this project is to develop a backend database for an e-commerce platform. The system will handle customer accounts, staff operations, product inventory, customer payment methods, and sales transactions.

2. User Roles

- **Customer:** An end user who creates an account, manages payment options, browses products, and places orders.
- **Staff:** An administrative user responsible for managing product inventory, updating product details, and monitoring purchase orders.

3. Data Requirements

Customer - Strong Entity

- **Attributes:** Customer ID (Primary Key), First Name, Last Name, Email, Password Hash, Phone Number, Date Joined
- **Rules:** Email must be unique. A customer may store multiple credit cards.

CreditCard - Weak Entity (dependent on Customer)

- **Attributes:** Card ID (Partial Key), Customer ID (Foreign Key), Card Number, Expiration Date, Cardholder Name, Billing Address
- **Rules:** Each card must belong to exactly one customer and cannot exist without one. A customer may store multiple cards. Participation: Customer side is partial and CreditCard side is total.

Staff - Strong Entity

- **Attributes:** Staff ID (Primary Key), First Name, Last Name, Email, Job Title, Hire Date
- **Rules:** Email must be unique. Staff members manage products and oversee order records.

Product - Strong Entity

- **Attributes:** Product ID (Primary Key), Product Name, Description, Price, Stock Quantity, Category
- **Rules:** Price must be greater than zero. Stock quantity updates when purchases are made. Each product is managed by a staff member.

Purchase - Strong Entity

- **Attributes:** Purchase ID (Primary Key), Customer ID (Foreign Key), Card ID (Foreign Key), Purchase Date, Total Amount, Status
- **Rules:** Every purchase must be associated with exactly one valid customer and exactly one stored payment card. A

purchase must contain one or more purchase items. Total Participation on Purchase side.

PurchaseItem - Weak Entity (dependent on Purchase and Product)

- **Attributes:** Purchase ID (Foreign Key), Product ID (Foreign Key), Quantity, Unit Price
- **Rules:** Composite primary key is (Purchase ID, Product ID). Captures individual products included in a specific purchase transaction. Cannot exist without both a parent Purchase and a referenced Product.

4. Relationships Summary

Relationship	Entities	Cardinality	Participation
Places	Customer – Purchase	1:M	Partial (Customer), Total (Purchase)
Stores	Customer – CreditCard	1:M	Partial (Customer), Total (CreditCard)
Manages	Staff – Product	1:M	Partial or Total (Staff, per business rule), Total (Product)
Contains	Purchase – PurchaseItem	1:M	Total (Purchase), Total (PurchaseItem)
Appears_in	Product – PurchaseItem	1:M	Partial (Product), Total (PurchaseItem)

5. System Use Cases

Use Case 1: Product Catalog Browsing

- **Actor:** Customer
- **Description:** Customers view available products, filter items by category, and search by product name or price.

Use Case 2: Product Inventory Management

- **Actor:** Staff
- **Description:** Staff members add new items to the inventory, update existing product details (such as price and description), and modify stock levels.

Use Case 3: Order Processing

- **Actor:** Customer
- **Description:** Customers select products, choose a saved payment method, and complete a purchase. The system records the transaction, itemizes each product in the purchase, and decrements product inventory.

Use Case 4: Order History

- **Actor:** Staff
- **Description:** Staff members view transaction history, order details, and sales totals across the system.

Running the Application

A terminal application for the e-commerce database: browse and manage the product catalog, place orders against a customer's saved credit card, and review customers and order history.

What you need

- **Python 3.10 or newer.** Check with `python --version` . If that command is not found, try `python3 --version` , and use `python3` everywhere below.
- **Nothing else.** We used SQLite, which is part of the Python standard library so there are no extra steps required.

Setup

Clone the repository and move into it:

```
git clone https://github.com/<user>/Dbase-Proj.git
```



```
cd Dbase-Proj
```



Create a virtual environment so the install does not touch your system Python.

On Windows:

```
python -m venv .venv && .venv\Scripts\activate
```



On macOS or Linux:

```
python3 -m venv .venv && source .venv/bin/activate
```



Install the one dependency:

```
pip install -r requirements.txt
```



Start it

```
python app.py
```



Press `q` to quit and `r` to reload the data in every tab.

Starting over

To wipe the database and reload the original sample data:

```
python app.py --reset
```



Running the SQL on its own

The queries the application uses are also collected in `queries.sql` so they can be run directly, without going through Python:

```
sqlite3 db/ecommerce.db ".headers on" ".mode column" ".read queries.sql"
```



Rele:

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors 2



[kee5625](#) [Karthik Rachamolla](#)



[KrishKalathiya](#) [Krish Kalathiya](#)

Languages

● Python 100%

Suggested workflows

Based on your tech stack



Python Package using Anaconda

Create and test a Python package on multiple Python versions using Anaconda for package management.

By GitHub Actions

[Configure](#)



Python application

Create and test a Python application.

By GitHub Actions

[Configure](#)



Pylint

Lint a Python application with pylint.

By GitHub Actions

[Configure](#)

[More workflows](#)

